

Ascend C 快速入门

目 录

1 引言：为什么需要 Ascend C	2
2 并行计算的基本原理	2
2.1 什么是并行计算	2
2.2 并行体系结构的分类	2
2.2.1 指令级并行的关键技术	2
2.2.2 线程级并行与请求级并行	3
2.2.3 数据级并行与任务级并行	3
2.3 Flynn 分类法	3
2.4 大模型并行：数据并行与模型并行	3
3 CANN 软件栈与开发环境	4
3.1 CANN 概述	4
3.2 CANN 软件包获取与安装	4
3.3 CANN 算子类型：AI Core 与 AI CPU	4
3.3.1 AI CPU 算子的适用场景	4
3.4 Runtime 运行时架构	5
3.4.1 进程中的任务调度	5
3.4.2 同步机制	5
4 核函数编写与调用	6
4.1 HelloWorld 核函数	6
4.2 核函数调用方式	6
4.2.1 内核符调用	6
4.2.2 单算子 API 调用	7
5 向量加法实战与加速比	7
5.1 核函数实现	7
5.2 加速比实测	9
5.3 CPU 仿真验证	9
6 本章你将学会	10
7 要点速查	10
8 小结	10

1 引言：为什么需要 Ascend C

深度学习训练与推理需要海量的矩阵运算，单靠 CPU 的串行计算远远不够。昇腾 AI 处理器通过 AI Core（人工智能核心）提供强大的并行加速能力，而 Ascend C 正是面向昇腾 AI Core 的算子开发编程语言。学习 Ascend C，意味着我们能够在硬件层面充分发挥昇腾的算力，为深度学习模型编写高效的定制算子。

直觉 不妨把 CPU 想象成一位精通各种任务的“全能工匠”，一件一件地处理工作；而 AI Core 则像一支由许多“专精工匠”组成的流水线队伍，同时处理成千上万个相同类型的数据。Ascend C 就是这支队伍的“指挥手册”，告诉每个工匠该拿哪些数据、做什么计算。

本章我们从并行计算的基本原理出发，理解 SPMD 编程模型和 Flynn 分类法；接着认识 CANN 软件栈与开发环境；然后通过 HelloWorld 和向量加法两个实例，掌握核函数的编写与调用方式；最后实测加速比，直观感受并行计算带来的性能提升。

2 并行计算的基本原理

2.1 什么是并行计算

并行计算（Parallel Computing）是一种同时执行多个计算任务或进程的计算模式。与串行计算按顺序逐一执行不同，并行计算让多个任务同时推进，以提高整体计算性能和效率。并行计算可以在多个硬件处理单元（如多个处理器、多个加速硬件、多个计算节点）上同时执行任务，有助于处理大规模的计算密集型问题，加快计算速度，提高系统吞吐量。

2.2 并行体系结构的分类

从不同视角出发，并行体系结构有多种分类方式。从计算机硬件、系统及应用层面来看，主要有三类：

1. **指令级并行（Instruction-Level Parallelism, ILP）**：处理器内部多个机器指令能在同一时钟周期执行，由处理器硬件自动管理，无需程序员手动优化。
2. **线程级并行（Thread-Level Parallelism, TLP）**：通过创建多个线程实现并行，多线程可真正并行地运行在不同核心上，常见于操作系统和数据库系统。
3. **请求级并行（Request-Level Parallelism, RLP）**：出现在应用服务中，服务器为不同客户端请求创建不同处理流程，同时处理多个独立请求。

从软件设计和编程模型的角度，又可划分为：

1. **数据级并行（Data-Level Parallelism, DLP）**：将大数据块分割成小块，在多个处理单元上并行执行相同操作，适合数组、向量和矩阵等数据结构。
2. **任务级并行（Task-Level Parallelism）**：将工作分解为独立任务，在不同处理单元上同时执行，任务间可能互相依赖也可能完全独立。

2.2.1 指令级并行的关键技术

ILP 有两种经典技术来提升并行度：

- **超标量架构（Superscalar）**：允许每个时钟周期发射多条指令到不同执行单元，如整数运算、浮点运算、加载/存储等，多个操作同时执行。

- **流水线 (Pipelining)**：将指令分解为小步骤，每个步骤由不同部件完成。一条指令的各阶段可与其它指令的阶段重叠。典型四段流水线包括取指 (IF)、译码 (ID)、执行 (EXE)、写回 (WB)。

流水线的直觉：就像工厂装配线，虽然每个产品要经过四道工序，但不同产品可以同时处于不同工序，从而整体吞吐量大幅提升。

2.2.2 线程级并行与请求级并行

TLP 需要程序员显式地通过编程来创建和管理线程，多线程在多核系统上真正并行运行，常见于操作系统、数据库系统及服务端应用。RLP 则常见于服务端，当多个独立客户端发送请求时，服务器创建不同处理流程并行响应，提高服务能力和响应速度。

2.2.3 数据级并行与任务级并行

DLP 特别适合数组、向量和矩阵等数据结构，常在科学计算和图像处理中使用。每个处理单元运行相同的操作，但作用于不同的数据片段。任务级并行需要程序员设计能有效利用并行硬件特性的算法，广泛应用于软件工程、复杂事件处理和多媒体应用。

直觉 昇腾 AI Core 的设计融合了上述多种并行层次：矩阵计算单元做大规模 DLP，多核之间做 TLP，核内流水线做 ILP。理解这些层次，有助于我们在编写算子时充分利用硬件特性。

2.3 Flynn 分类法

Michael Flynn 于 1972 年提出了一种经典的体系结构分类法，以指令流和数据流的数量组合为依据，将计算机系统划分为四类：

分类	全称	指令流	数据流
SISD	单指令流单数据流	单	单
SIMD	单指令流多数据流	单	多
MISD	多指令流单数据流	多	单
MIMD	多指令流多数据流	多	多

传统 CPU 属于 SISD/SIMD 混合架构，GPU 属于 SIMT（类 SIMD），而昇腾 AI Core 的向量计算单元本质上也是 SIMD 模式，矩阵计算单元则是更深层次的批量数据并行。

此外，还有更复杂的并行处理模式 **SPMD (Single Program Multiple Data, 单程序多数据)**。当硬件的各处理器有独立控制部件时，可通过软件编程让各处理器并行执行同一个程序，但每个处理器处理不同的数据。Ascend C 就是基于昇腾 AI Core 形成的 SPMD 编程模型：核函数编写一次，启动时指定 block 数量，每个 block 通过 `block_idx` 区分自己负责的数据分片。这与 CUDA 的网格/块模型非常相似。

2.4 大模型并行：数据并行与模型并行

针对 AI 大模型的并行化训练，主要常用两类并行方式：

1. **数据并行 (Data Parallelism)**：将大规模数据集划分为多个批次，分配给不同计算节点并行处理。各节点持有模型副本，训练后通过 AllReduce 同步梯度。数据并行使训练可扩展到更多数据和计算资源，从而加速训练过程。
2. **模型并行 (Model Parallelism)**：当单个模型太大而无法放入单节点内存时，将模型的不同部分（如不同层或子网络）分布到不同节点。各节点负责模型的一部分计算，需频繁跨节点通信以同步中间状态和梯度信息。模型并行又可分张量并行和流水线并行。

Ascend C 算子开发更关注的是单设备内的数据并行，即如何把大批量数据切分给 AI Core 的多个核心并行处理。

3 CANN 软件栈与开发环境

3.1 CANN 概述

CANN (Compute Architecture for Neural Networks, 神经网络计算架构) 是华为昇腾异构计算架构的软件栈，连接上层深度学习框架与底层昇腾硬件。CANN 提供从算子开发、模型转换到运行时管理的完整工具链。

CANN 的核心层次包括：

1. **Ascend C**：面向 AI Core 的 C++ 扩展算子编程语言。
2. **Graph Engine (图引擎)**：将计算图编译为可在昇腾设备上执行的任务。
3. **Runtime (运行时)**：管理设备、内存、流、事件等底层资源。
4. **ACL (Ascend Computing Language)**：提供 C 语言接口供 Host 程序调用。
5. **ATC (模型转换工具)**：将 ONNX、Caffe 等框架模型转为昇腾离线模型。

3.2 CANN 软件包获取与安装

CANN 软件包可从昇腾社区官网 (<https://www.hiascend.com>) 获取。下载时推荐选择“社区版”，并选择与硬件匹配的版本。安装前需确认第三方依赖已就绪，随后通过命令行完成安装。安装完成后可查阅发布版本信息 (Release Note) 了解版本特性与已知问题。

具体安装步骤请参考昇腾社区官方文档，不同版本和操作系统（如 Ubuntu、CentOS、EulerOS）的安装命令略有差异。

3.3 CANN 算子类型：AI Core 与 AI CPU

昇腾设备上的 CANN 算子分为两大类：

- **AI Core 算子**：运行在 AI Core 上，执行数据密集的矩阵、向量及标量运算。矩阵乘、卷积、激活函数等高吞吐计算都属于此类，是 Ascend C 的主要开发目标。
- **AI CPU 算子**：运行在 AI CPU 上，执行不适合在 AI Core 中运行的算子，即非矩阵类复杂计算。

3.3.1 AI CPU 算子的适用场景

在以下三种情况下，可以使用 AI CPU 方式实现自定义算子：

1. 不适合跑在 AI Core 上的算子，例如非矩阵类的复杂计算、逻辑复杂的分支密集型算子，如离散数据计算、资源管理类计算、依赖随机数生成的计算。
2. AI Core 不支持的算子，例如算子需要 Complex32、Complex64 等数据类型但 AI Core 不支持。
3. 为快速打通模型执行流程，在 AI Core 实现较困难时，先通过 AI CPU 算子进行功能调测，功能调通后再转换为 AI Core 算子实现以提升性能。

AI CPU 算子编译执行涉及的组件包括 GE（Graph Engine）、Data Processor、AI CPU Engine、AI CPU Schedule 和 AI CPU Processor，最终在 AI CPU 上执行。

3.4 Runtime 运行时架构

Runtime（运行时） 为神经网络的任务分配提供资源管理通道，运行在应用程序的进程空间中。它提供以下核心功能：

概念	说明
Memory	存储管理，包括 Host 内存与 Device 内存之间的拷贝
Device	代表一个昇腾 NPU 设备
Stream	执行流，同一 Stream 内的操作按序串行执行
Event	事件，用于跨 Stream 同步
Kernel	核函数，在 Device 上并行执行的计算单元

Task Schedule（任务调度） 运行在 Device 侧的任务调度 CPU 上，负责将 Runtime 分发的具体任务进一步分发到 AI CPU 上。

3.4.1 进程中的任务调度

进程中的任务调度涉及两个过程：

1. **流创建过程**：流在进程内向上提供保序队列的抽象，每个进程在每个 Device 上可独立创建一条或若干条流。
2. **任务下发调度过程**：由 Kernel 完成，是 Runtime 和 Task Scheduler 的异步流水过程，建立在保序队列基础上。

3.4.2 同步机制

Runtime 的同步机制分为三类：

1. **Stream 间同步**：协调不同流之间的执行顺序。
2. **Task 与 Host 同步**：Device 侧任务与 Host 侧程序的同步。
3. **Stream 与 Host 同步**：流与 Host 侧程序的同步。

同一 Stream 内的操作是串行执行的，不同 Stream 之间可通过同步机制协调。

直觉 把 Stream 想象成超市的收银队伍：同一队伍内顾客按先后顺序结账（串行），不同队伍之间互不影响（并行），但有时需要等某个队伍完成特定任务后才能继续（同步）。

4 核函数编写与调用

4.1 HelloWorld 核函数

Ascend C 核函数是运行在 AI Core 上的 C++ 函数。以下是一个最简单的 HelloWorld 示例：

```
#include "kernel_operator.h"

using namespace AscendC;

extern "C" __global__ __aicore__ void hello_world() {
    PRINTF("Hello World from block %d\n", GetBlockIdx());
}
```

代码逐行解读：

1. `#include "kernel_operator.h"`：引入 Ascend C 算子开发的核心头文件，包含所有内置 API。
2. `using namespace AscendC;`：使用 AscendC 命名空间，省去每次调用 API 时加前缀。
3. `extern "C"`：以 C 链接方式导出函数名，防止 C++ 名称修饰导致 Host 端找不到符号。
4. `__global__`：表示这是一个可从 Host 端调用的核函数。
5. `__aicore__`：表示该函数运行在 AI Core 上。
6. `PRINTF(...)`：在核函数中输出信息，`GetBlockIdx()` 返回当前 block 的编号。

例 假设启动 8 个 block，则 `GetBlockIdx()` 分别返回 0 到 7，每个 block 打印“Hello World from block 0”、“Hello World from block 1”一直到“Hello World from block 7”。这就是 SPMD 模型：同一份代码，8 个 block 并行执行，各自通过 `GetBlockIdx()` 区分自己。

4.2 核函数调用方式

Ascend C 算子核函数的调用方式有 2 种：**内核符调用**和**单算子 API 调用**。内核符调用方式通常在算子的快速开发中使用，简单直接，主要目的是检验算子的正确性。单算子 API 调用则要求算子已通过完整开发流程进行编译部署，在正式部署场景中使用。

4.2.1 内核符调用

内核符调用使用 `<<<...>>>` 语法，分为 NPU 模式和 CPU 仿真模式两种执行方式。

NPU 模式 用于上板执行，在核函数调用应用程序中进行内核符调用：

```
constexpr int32_t BLOCK_NUM = 8;
hello_world<<<BLOCK_NUM>>>(stream);
```

这里 `BLOCK_NUM = 8` 指定启动 8 个 block，`stream` 指定执行流。每个 block 并行执行 `hello_world` 函数体。

CPU 模式 用于仿真，验证代码逻辑正确性，使用 `ICPU_RUN_KF` 接口直接调用核函数：

```
ICPU_RUN_KF(hello_world, 8);
```

CPU 模式下可使用 `gdb` 断点调试和 `printf` 打印变量，是开发初期最常用的调试手段。

4.2.2 单算子 API 调用

单算子 API 调用 (ACLNN) 在算子经过完整开发流程、编译为 .so 库后使用。典型调用流程如下：

```
aclrtMalloc(&xDevice, byteSize, ACL_MEM_MALLOC_HUGE_FIRST);
aclnnAddCustomGetWorkspaceSize(...);
aclnnAddCustom(...);
aclrtFree(xDevice);
```

各步骤含义：先在 Device 上分配内存 (aclrtMalloc)，再查询算子所需 workspace 大小 (aclnnAddCustomGetWorkspaceSize)，然后执行算子 (aclnnAddCustom)，最后释放内存 (aclrtFree)。这种方式适合正式部署场景。

5 向量加法实战与加速比

5.1 核函数实现

以下是一个完整的向量加法核函数，演示 SPMD 模型和基本的向量编程范式。我们以 half 类型为例，实现 $z = x + y$ 。

```
#include "kernel_operator.h"

using namespace AscendC;

constexpr int32_t BUFFER_NUM = 2;
constexpr int32_t TILE_LENGTH = 128;

template <typename T>
class KernelAdd {
public:
    __aicore__ inline KernelAdd() {}
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z,
                                uint32_t totalLength) {
        ASSERT(GetBlockNum() != 0 && "block dim can not be zero");
        this->blockLength = totalLength / GetBlockNum();
        this->tileNum = blockLength / TILE_LENGTH;
        xGm.SetGlobalBuffer((__gm__ T*)x
                             + this->blockLength * GetBlockIdx(), this->blockLength);
        yGm.SetGlobalBuffer((__gm__ T*)y
                             + this->blockLength * GetBlockIdx(), this->blockLength);
        zGm.SetGlobalBuffer((__gm__ T*)z
                             + this->blockLength * GetBlockIdx(), this->blockLength);
        pipe.InitBuffer(inQueueX, BUFFER_NUM, TILE_LENGTH * sizeof(T));
        pipe.InitBuffer(inQueueY, BUFFER_NUM, TILE_LENGTH * sizeof(T));
        pipe.InitBuffer(outQueueZ, BUFFER_NUM, TILE_LENGTH * sizeof(T));
    }
    __aicore__ inline void Process() {
        int32_t loopCount = this->tileNum;
        for (int32_t i = 0; i < loopCount; i++) {
            CopyIn(i);
            Compute(i);
        }
    }
};
```



```

        CopyOut(i);
    }
}
private:
__aicore__ inline void CopyIn(int32_t progress) {
    LocalTensor<T> xLocal = inQueueX.AllocTensor<T>();
    LocalTensor<T> yLocal = inQueueY.AllocTensor<T>();
    DataCopy(xLocal, xGm[progress * TILE_LENGTH], TILE_LENGTH);
    DataCopy(yLocal, yGm[progress * TILE_LENGTH], TILE_LENGTH);
    inQueueX.Enqueue(xLocal);
    inQueueY.Enqueue(yLocal);
}
__aicore__ inline void Compute(int32_t progress) {
    LocalTensor<T> xLocal = inQueueX.DeQueue<T>();
    LocalTensor<T> yLocal = inQueueY.DeQueue<T>();
    LocalTensor<T> zLocal = outQueueZ.AllocTensor<T>();
    Add(zLocal, xLocal, yLocal, TILE_LENGTH);
    outQueueZ.Enqueue<T>(zLocal);
    inQueueX.FreeTensor(xLocal);
    inQueueY.FreeTensor(yLocal);
}
__aicore__ inline void CopyOut(int32_t progress) {
    LocalTensor<T> zLocal = outQueueZ.DeQueue<T>();
    DataCopy(zGm[progress * TILE_LENGTH], zLocal, TILE_LENGTH);
    outQueueZ.FreeTensor(zLocal);
}
};

extern "C" __global__ __aicore__ void add_custom(
    GM_ADDR x, GM_ADDR y, GM_ADDR z,
    GM_ADDR workspace, GM_ADDR tiling) {
    KernelAdd<half> op;
    uint32_t totalLength = *((__gm__ uint32_t*)tiling);
    op.Init(x, y, z, totalLength);
    op.Process();
}

```

这段代码的结构是 Ascend C 算子的标准范式，我们来逐段理解。类中还有 TPipe、TQue、GlobalTensor 等成员变量，此处省略声明，重点看逻辑。

Init 函数：初始化与数据切分

1. GetBlockNum() 获取启动的 block 总数，GetBlockIdx() 获取当前 block 的编号。
2. blockLength = totalLength / GetBlockNum(): 将总数据量均分给每个 block。例如总长 10240、8 个 block，则每个 block 处理 1280 个元素。
3. tileNum = blockLength / TILE_LENGTH: 每个 block 内部再将数据切分为若干 tile，每次处理一个 tile。例如 1280 / 128 = 10 个 tile。
4. SetGlobalBuffer: 为 xGm、yGm、zGm 设置 **Global Memory（全局内存）** 的起始地址，每个 block 只访问自己负责的那段数据，偏移量为 blockLength * GetBlockIdx()。

5. pipe.InitBuffer: 为输入队列 inQueueX、inQueueY 和输出队列 outQueueZ 分配 **Local Memory (局部内存)** 空间, BUFFER_NUM = 2 表示双缓冲, 实现搬运与计算的流水重叠。

Process 函数: 三段式流水循环

Process 循环执行 tileNum 次, 每次完成三个步骤:

1. CopyIn: 从 Global Memory 搬运数据到 Local Memory (DataCopy), 并将 tensor 入队 (EnQueue), 通知计算单元数据就绪。
2. Compute: 从队列取出数据 (DeQueue), 调用 Add 执行向量加法, 将结果入输出队列。
3. CopyOut: 从输出队列取出结果, 搬回 Global Memory。

这种 **CopyIn-Compute-CopyOut** 三段式流水是 Ascend C 向量算子的核心编程范式, 配合双缓冲实现搬运与计算的并行。

例 取小数值算给你看: 假设 totalLength = 8, 启动 BLOCK_NUM = 2 个 block, TILE_LENGTH = 2。

- blockLength = 8 / 2 = 4: block 0 处理元素 0,1,2,3, block 1 处理元素 4,5,6,7。
- tileNum = 4 / 2 = 2: 每个 block 分 2 次处理, 每次 2 个元素。

以 block 0 为例: 第 1 次循环 CopyIn 搬入元素 0 和 1, Compute 计算 $z[0] = x[0] + y[0]$ 、 $z[1] = x[1] + y[1]$, CopyOut 搬回结果; 第 2 次循环处理元素 2 和 3。block 1 同时进行类似操作处理元素 4,5,6,7。两个 block 并行执行, 总耗时约为单个 block 的一半。

5.2 加速比实测

在 Ascend 910B1 上, 对 10240 个 float 数据执行向量加法, 对比 NPU 加速处理与 CPU 串行处理的时间开销:

实现方式	耗时 (μ s)	说明
CPU 串行	约 1200	单核循环加法
Ascend C 核函数	约 8	8 个 block 并行

加速比约 **150 倍**, 直观展示了 SPMD 并行模型在大规模数据上的优势。实际加速比取决于数据量、数据类型和 block 数量的配置。

直觉 加速比远超 block 数量 (8 个 block 却获得约 150 倍加速) 的原因在于: AI Core 的向量计算单元本身是一条 SIMD 流水线, 每个 block 内部的 Add 操作本身就是并行的; 再加上多 block 并行和双缓冲流水, 三重并行叠加才带来如此高的加速比。

5.3 CPU 仿真验证

在正式上板运行前, 先用 CPU 仿真模式验证逻辑正确性:

```
ICPU_RUN_KF(add_custom, blockDim, x, y, z, workspace, &tiling);
```

仿真模式下, 核函数在 CPU 上执行, Add 等向量操作由仿真库模拟。开发者可以用 printf 打印中间结果, 或用 gdb 设断点逐行调试, 确认逻辑无误后再上板验证性能。

6 本章你将学会

1. 理解并行计算的五种层次（ILP、TLP、RLP、DLP、任务级）及 Flynn 分类法，能判断不同硬件属于哪种架构。
2. 掌握 SPMD 编程模型的核心思想，理解 Ascend C 如何通过 `block_idx` 实现一程序多数据。
3. 认识 CANN 软件栈的层次结构，区分 AI Core 算子与 AI CPU 算子的适用场景，理解 Runtime 的核心概念（Device、Stream、Event、Kernel）。
4. 能够编写 HelloWorld 和向量加法核函数，掌握 `__global__`、`__aicore__`、`GetBlockIdx()` 等关键语法。
5. 掌握两种核函数调用方式（内核符调用 `<<<...>>>` 和单算子 API 调用 ACLNN），以及 CPU 仿真调试方法。

7 要点速查

概念	要点
并行计算层次	ILP、TLP、RLP（硬件层）；DLP、任务级（软件层）
Flynn 分类	SISD、SIMD、MISD、MIMD, 1972 年提出
SPMD	单程序多数据, Ascend C 核心编程模型
大模型并行	数据并行（切数据）、模型并行（切模型）
CANN 算子类型	AI Core 算子（矩阵/向量）、AI CPU 算子（非矩阵复杂计算）
核函数修饰符	<code>__global__</code> （Host 可调用）、 <code>__aicore__</code> （运行在 AI Core）
调用方式	内核符 <code><<<...>>></code> （快速验证）、ACLNN（正式部署）
仿真调试	ICPU_RUN_KF, 配合 gdb 和 printf
三段式流水	CopyIn → Compute → CopyOut

8 小结

本章从“为什么需要 Ascend C”出发，介绍了并行计算的基本原理，包括五种并行层次和 Flynn 分类法，理解了 SPMD 编程模型是 Ascend C 的核心。我们认识了 CANN 软件栈的层次结构，区分了 AI Core 算子与 AI CPU 算子，并了解了 Runtime 的流、事件和同步机制。通过 HelloWorld 和向量加法两个实例，读者初步掌握了核函数的编写方式和两种调用模式，实测加速比约 150 倍展示了并行计算的威力。

后续章节将深入编程模型、开发流程、调试调优和大模型算子优化等主题，我们将在此基础上逐步构建更复杂的算子。